



Transcript

Week 13: Recommendation Engine

Video 1 - Recommendation Engine

How do apps know exactly what you might like next—whether it's a movie, a product, or a song? Behind the scenes, recommendation engines are analysing patterns and making smart predictions. Let's learn how these systems work and why they matter in today's digital world.

In today's digital world, we're surrounded by choices—from movies and music to clothes and books. Recommendation engines help us cut through the noise by suggesting items that match our preferences. These systems act like personal assistants. They use data about what we watch, buy, or listen to—and compare it with others' behaviour—to predict what we might enjoy next. Whether it's Netflix offering a new show or Amazon recommending a product, these tools make online experiences more personal and useful.

Recommendation engines are powerful tools that personalise our digital experience. They use different methods to suggest content based on what users like.

The first type is **collaborative filtering**, which relies on data from other users. In **user-based collaborative filtering**, the system looks for users with similar preferences and suggests items they enjoyed. In **item-based collaborative filtering**, it recommends items similar to ones you've liked in the past.

The second type is **content-based filtering**. This method focuses on the features of an item—such as genre, topic, or description—and compares them to what you've liked before.

Finally, **hybrid systems** combine both collaborative and content-based methods. This improves the accuracy and variety of recommendations by using both user behaviour and item details.

Want smarter suggestions, faster?

Learn how recommendation engines work—and how you can use them to personalise your world. Let's get started!

Video 2 - Collaborative Filtering

Have you ever noticed how apps recommend things you didn't search for—but still like? That's collaborative filtering at work. It finds users with similar preferences and suggests what they've enjoyed. Let's learn how this method helps personalise your online experience.

Collaborative filtering is a technique used in recommendation systems to suggest items based on user preferences. It works by identifying patterns—either between users or between items.

In **user-based filtering**, the system finds users who rate things similarly. If you and another user both rated the same movies highly, the system assumes you have similar tastes and recommends items that the other person liked.

In **item-based filtering**, the focus is on similarities between items. For example, if you liked the movie *Inception*, the system might recommend *The Matrix* because many users



who liked one also liked the other.

Collaborative filtering performs well when there's a large dataset of users and item ratings. But it struggles with the **cold-start problem**. This occurs when there's not enough data—such as when a **new user** joins or a **new item** is added. With little or no rating history, the system can't generate reliable recommendations.

To overcome this, many platforms use hybrid models that combine collaborative filtering with other methods, like content-based filtering.

User-based collaborative filtering is a method used by recommendation systems to suggest items to a user by finding other users with similar tastes. The idea is simple: if two users have liked the same things in the past, they will probably enjoy similar things in the future. Based on this, the system recommends items liked by one user to another who has similar preferences. This approach relies on the behaviour and ratings of like-minded users to make personalised suggestions. User-based collaborative filtering finds users with similar preferences to the target user and recommends items they've liked. This is done by comparing user behaviour, like ratings or purchase history.

To measure similarity, systems use metrics such as **cosine similarity**, which calculates the angle between two users' rating patterns; **Pearson correlation**, which measures how closely the users' preferences match; and **Jaccard similarity**, which compares shared ratings using liked or disliked items.

Each user has a profile based on their interactions with different items. These profiles are often modelled as vectors in a multi-dimensional space, with each item as a separate dimension.

After identifying similar users, the system forms a "neighbourhood" by selecting the top few most similar users. The number of neighbours chosen can affect the quality of recommendations. A well-formed neighbourhood improves the system's accuracy and personalisation.

Once the system identifies users with similar preferences, the next step is to predict how the target user might rate an item they haven't interacted with yet. This is done using a formula that combines the ratings from similar users—also called neighbors—with how similar they are to the target user. Each neighbor's rating is given a weight based on their similarity. All these weighted ratings are then summed up and normalized by the total similarity scores. This process ensures that more relevant users have a greater impact, resulting in a balanced and accurate prediction. After predicting how a user might rate items they haven't seen, the system generates recommendations. It ranks all items based on their predicted ratings, calculated using the input of similar users.

Then, it suggests the top-rated items that the user hasn't interacted with yet. These recommendations are tailored to the user's interests, making the suggestions more relevant and engaging.

This personalised approach helps increase user satisfaction and keeps content discovery meaningful.

However, it's important to keep refining the system to address challenges like the cold-start problem, where there's not enough data for new users or items, and data sparsity, where user-item interactions are limited. Tackling these issues helps ensure that recommendations stay accurate and useful over time.

In collaborative filtering, memory-based methods rely on direct comparisons between users. One common approach is user-user collaborative filtering. It works on the idea that if two users liked the same things before, they'll likely agree again. The system finds

users with similar preferences and recommends items one user liked to the other. This method is simple and intuitive, making it easy to implement. However, as the number of users and items grows, it can become difficult to scale. For example, someone who enjoys action movies might be recommended a film like *Inception* because other users with similar tastes rated it highly.

Model-based collaborative filtering uses machine learning to predict what users might like. Instead of just comparing users directly, it builds models to understand user behavior and item relationships. Techniques like clustering group users with similar preferences. Matrix factorization, such as SVD, breaks down large user-item data into simpler patterns. And deep learning can capture more complex behavior for even smarter recommendations. These methods are powerful and can scale to large datasets while revealing hidden patterns. But they can also be more complex to build and need more computing power.

Hybrid collaborative filtering combines different recommendation techniques to improve accuracy. It often merges user-based collaborative filtering with content-based filtering. This way, the system learns from both user behavior and item features. The strength of this approach is that it balances multiple methods, making recommendations more diverse and personalized. It also helps solve common challenges like the cold-start problem. For example, a user might get suggestions based on what they've watched before and also on the content of those items—ensuring the results are both relevant and tailored.

Using different collaborative filtering methods helps create smarter, more personalized recommendations—leading to better user engagement and satisfaction.

User-based collaborative filtering offers several key advantages. First, it provides personalised recommendations based on users with similar preferences. This allows the system to suggest content that aligns with the unique tastes of each user.

Second, it doesn't require any prior knowledge of item attributes, such as genre or description. This makes it highly adaptable across different domains, like movies, books, or music, where item details may be unavailable or inconsistent.

Lastly, this method reveals useful insights into user behaviour. By observing how users interact with different items, the system can detect trends and shifts in preferences—valuable information for marketers, content creators, and platform designers.

When it comes to making great recommendations, item-based collaborative filtering takes the spotlight by focusing on item relationships—rather than user similarities. Imagine you've enjoyed a particular movie or product. Instead of finding others like you, this method looks for items that are similar to what you already liked. If those items are rated similarly by other users, chances are, you'll like them too. By analyzing patterns across ratings and interactions, the system figures out which items go well together. Then it uses this insight to suggest new options tailored to your past behavior. This technique is a favorite in e-commerce and streaming platforms, where knowing what items click together boosts satisfaction and keeps users engaged.

Let's walk through a simple example of how user-based collaborative filtering works.

First, the system identifies users who have rated items similarly to the target user—User A. For instance, if User B and User C gave similar ratings to movies like *Inception* and *Titanic*, they are considered similar to User A.

To determine this, the system uses similarity measures such as cosine similarity or Pearson correlation. These methods compare the users' rating patterns mathematically to find matches.



Next, the system predicts how User A might rate a movie they haven't seen yet—like *Toy Story*. It looks at how similar users rated that movie:

User B gave it a 3, and User C gave it a 2. Based on this, the system estimates that User A would likely rate *Toy Story* somewhere close to the average of these ratings.

This prediction helps generate a personalised recommendation for User A.

Once the system has identified users similar to the target user, it can estimate how the target user might rate an item they haven't seen.

In this example, User A hasn't rated *Toy Story*, but similar users—User B and User C—have. User B gave it a rating of three, and User C gave it a two.

To predict how User A might rate *Toy Story*, the system takes the average of these two ratings. That is, three plus two, divided by two, which equals two point five.

So, the system predicts that User A would likely give *Toy Story* a rating of two point five and recommends the item accordingly.

Can you think of a platform you use where similar users might be influencing your recommendations—without you even realising it?

Video 3 - Content-Based Filtering

What if your next favorite thing was just like your last? That's the magic of content-based filtering.

Content-based filtering recommends items by focusing on the features of the items themselves. It analyses what a user has previously liked—such as genre, topic, or keywords—and then suggests other items with similar characteristics.

This method doesn't depend on other users' preferences. Instead, it builds a profile for each user and recommends items that match their own past interactions. It's particularly useful when there's limited data about other users, or when a system needs to tailor suggestions based solely on individual preferences.

In content-based filtering, each item is described by a set of features—such as genre, director, or cast for movies, or keywords, category, and author for articles. These are called **item attributes**.

The system builds a **user profile** based on items the user has previously interacted with—liked, rated, or purchased. This profile highlights the features most relevant to that user.

Next, the system performs a **similarity calculation** between the user's liked items and the rest of the catalogue. Techniques like **cosine similarity** or **Euclidean distance** are used to measure how closely items match based on their attributes.

Finally, the system generates a **recommendation** by selecting new items that are most similar to those the user already enjoyed. This helps deliver personalised, relevant suggestions aligned with the user's preferences.

Let's look at how content-based filtering works in a movie recommendation system. The system learns from User A's viewing history. For example, User A has liked "Inception" and "Interstellar." Both movies are science fiction, directed by Christopher Nolan, but feature different lead actors—Leonardo DiCaprio and Matthew McConaughey. The system

then examines the attributes of these liked movies—such as genre, director, and actors—to find patterns. Based on these patterns, it recommends other movies that share similar features. This approach helps personalise recommendations using only the content of previously liked items.

After identifying the movies User A likes, the system creates a user profile. It notes that User A prefers science fiction, enjoys films directed by Christopher Nolan, and is likely to watch movies featuring actors like Matthew McConaughey and Leonardo DiCaprio. Using this profile, the system searches for other movies in its catalogue that match these attributes. For example, it might recommend *The Prestige*, which is also directed by Nolan, or *Gravity*, which stars McConaughey. These suggestions are based on how closely the new movies match the features of the ones User A has already liked.

Content-based filtering has some strong advantages. It gives users **personalised recommendations** based on what they've liked before. It can even suggest new items without needing user interactions first—as long as the system knows the item's features. Another benefit is **transparency**—users can easily understand why certain recommendations are made. However, there are also some weaknesses. One issue is **limited diversity**. The system might keep recommending similar content, creating a “filter bubble.” It also struggles with new users who have no history, making it harder to generate relevant suggestions early on.

Content-based filtering is used across different industries. In streaming services, it recommends movies or shows based on what users have previously watched and liked—using details like genre, director, or actor. In e-commerce, the system suggests products that are similar to what a user has bought or viewed before. For news or articles, it recommends content based on what the user has read, often by matching keywords. In all cases, the system focuses on finding items with similar features to what the user already prefers.

Content-based filtering works by recommending items based on their features, not user behaviour. One key step in this process is feature extraction. This means identifying important details about each item—such as genre, director, or actors in a movie, or keywords in an article. These features are then used to compare items and make recommendations that match the user's preferences. The system builds a profile for each user based on what they have liked before and uses it to suggest similar items. This ensures the recommendations are relevant and personalised.

The first step in content-based filtering is identifying the features of an item that will help make accurate recommendations. These features vary depending on the type of content. For text like books or articles, useful features include the title, author, genre, keywords, and a short summary. For multimedia content like images or videos, important features include visual attributes such as colour, texture, and shape, as well as metadata like tags or descriptions. In e-commerce, product features may include the brand, category, price, and technical specifications. By identifying and extracting these features, the system can create a strong framework for making personalised recommendations that match the user's interests.

The second step in feature extraction is data preprocessing. This step prepares the data for analysis by cleaning and transforming it into a format that the system can work with. One key process is **normalisation**, which scales numerical features so they contribute equally during analysis. Another is **encoding**, which turns text labels like categories into numbers—for example, by using one-hot encoding. For text data, we apply **text processing** steps like **splitting text into words** or tokenisation, **removing common words** like “and” or “the,” and **reducing words to their base form** using techniques like stemming or lemmatisation—for instance, changing “running” to “run.” We can also use a statistical measure called **term frequency-inverse document frequency**, or T-F-I-D-F, to identify the most important words in a document. These steps help ensure the

data is clean, consistent, and ready for generating accurate recommendations.

Feature representation is a critical phase in content-based filtering. This is where we choose how to represent the features of an item in a way that captures its meaning. These representations allow the system to compare items and make accurate recommendations.

One common method is **vectorisation**, where items are turned into vectors in a multi-dimensional space. Each dimension corresponds to a feature, making it easier to analyse and compare content—especially useful for text data.

Another method is **Bag of Words**, which counts how many times each word appears in a document. It ignores grammar and word order but still gives a basic, useful structure for comparing text.

We can also use **TF-IDF**, or Term Frequency–Inverse Document Frequency. This method builds on the Bag of Words model by giving more weight to words that are unique or important in a document, and less to common words that appear frequently across the dataset.

Finally, there's **word embeddings**, which use models like Word2Vec or GloVe. These convert words into dense numerical vectors that capture deeper meaning and relationships between words. This makes it possible for the system to understand not just the words themselves, but the context and meaning behind them.

By using these different methods of feature representation, content-based filtering systems can better understand the data and deliver more accurate, personalised recommendations.

Feature selection is a crucial step in the content-based filtering process. Here, we identify the most relevant features that will improve the performance of the recommendation system. By focusing on the features that matter most, we can enhance both the accuracy and efficiency of the model.

One approach is **univariate selection**, where each feature is evaluated independently to see how strongly it relates to the target outcome. This helps identify which features are useful for generating recommendations and allows us to filter out less important ones.

Another method is **recursive feature elimination**, or **RFE**. This is an iterative process where we remove one feature at a time and measure how it affects the model's performance. Features that don't have much impact can be removed, leaving behind only those that contribute meaningfully to the system's predictions.

A third technique is **Principal Component Analysis**, or **PCA**. PCA is a dimensionality reduction method. It transforms the original features into a smaller set of uncorrelated variables called principal components. These components retain the most important information in the dataset while reducing complexity.

By using these feature selection techniques, content-based filtering systems can optimise their feature sets. This makes the model more efficient and leads to more relevant, personalised recommendations for users.

Once items are represented by their features, the next important step in content-based filtering is to measure how similar they are. Measuring similarity helps the system recommend items that are alike based on their features.

One commonly used method is **cosine similarity**, which measures the angle between two item vectors in a feature space. A score of 1 means the items are very similar, while



0 means no similarity at all. Cosine similarity is especially useful for high-dimensional data, such as text represented as word frequencies.

Another approach is **Euclidean distance**, which calculates the straight-line distance between two points in feature space. The smaller the distance, the more similar the items. Since it depends on the scale of features, data often needs to be normalised first.

Finally, **Jaccard similarity** is used to compare sets of features. It looks at how much two sets overlap, compared to their total size. This method is especially helpful for binary data, like checking whether certain features are present or not.

By using these similarity measures, content-based filtering systems can recommend items that are closely aligned with a user's preferences, improving the overall experience.

As you reflect on similarity measurement techniques, consider this: How would you choose the most appropriate method for comparing items in a content-based system—like course recommendations or academic papers—where accuracy and relevance really matter?

Video 4 – Market Basket Analysis

Welcome to this session on market basket analysis for recommendation system. In the world of retail, e-commerce and digital platforms, understanding customer behaviour is key. One of the most powerful techniques to uncover such behaviour is market basket analysis, also known as association rule mining.

It helps us answer the question if a customer buys x , what are they likely to buy next. This method originally rooted in transactional data like supermarket purchases is now being applied extensively in modern recommendation system from suggesting related products on Amazon to recommending playlists on Spotify or even bundling services in telecom and insurance. In this module, we will explore how patterns of co-purchase behaviour are extracted using algorithms like Apriori and how support confidence and lift metrics guide us in selecting meaningful association.

We'll also look at how these insights can be operationalised into personalised recommendation using rule-based engines. By the end of this lesson, you will not only understand the math and logic behind market basket analysis, but also how it translates into real-world AI-driven recommendations that impact user experience and business revenue. Let's dive in.

So, what exactly is market basket analysis? At its core, it's a data-driven technique used to identifying purchasing patterns, that is, which items tend to be bought together. Imagine scanning a thousand shopping carts and noticing that customers who buy bread also frequently buy butter. That's pattern recognition.

But it doesn't stop there. Association discovery goes deeper. It uncovers hidden relationships between products that may not be obvious to the human eye.

For example, customer buying diapers might also be purchasing beer. Not because they go together logically, but because of a hidden behavioural pattern. The third element is probability analysis.

Market basket analysis helps us calculate the likelihood of these items' combinations. It



answers questions like, what is the probability that a customer who buys item A also buys item B? This is quantified using metrics like support, confidence, and lift, which we will cover in more detail soon. Together, these three components empower businesses to drive smarter recommendations, design effective promotions, and personalise experiences, all based on what customers actually do, not just what they say.

Now that we understand what market basket analysis is, let's explore why it plays such an important role in recommendation systems. First, MBA offers a complementary approach, which collaborative filtering relies on user behaviour and content-based filtering focusses on product features. MBA uses co-occurrence pattern in transactions.

When combined, these approaches produce more well-rounded recommendations, enhancing both relevance and diversity. Second, market basket analysis is transaction-driven. This means it works with actual purchase behaviour, not just what users rate or click on.

It captures real-world intent, which items were bought together, making it more trustworthy and behaviourally grounded. Third, and perhaps most compelling for businesses, is the direct business value. Market basket analysis helps increase the average basket size by revealing smart cross-sell and up-sell opportunities.

Think about frequently bought together suggestions on Amazon. That's market basket analysis in action, and it translates into measurable revenue gains. When integrated thoughtfully, market basket analysis can make your recommendation system smarter, more grounded, and more profitable.

Let's now talk about the first and most fundamental metric used in market basket analysis, support. Support tells us how frequently an item or a set of items appears in our dataset. It is calculated as the proportion of transactions that contain a specific item set compared to the total number of transactions.

Mathematically, it is expressed as transactions containing A divided by total transactions when we are calculating support for the item A. For example, if we have 1000 total transactions and 100 of them include both bread and butter, the support for the item set bread and butter would be equal to 100 divided by 1000, which is equals 10%. Support helps us answer the question, how common is this combination? Which is useful for identifying frequent patterns. In practise, we often set a minimum support threshold to filter out rare item sets and focus on those with meaningful frequency.

The higher the support, the more frequently that combination appears and the more valuable it becomes in making reliable recommendations. Let's now move to the next important metric in market basket analysis, confidence. While support tells us how often items occur together, confidence helps us understand how reliable a rule is for making predictions.

Specifically, if it answers the question, if a customer buys item A, how likely are they will also buy item B? The formula for the confidence is $A \text{ implies } B$ means people when they are buying A, they purchase B also and the confidence formula is support of A and B and divided by support of A. This means we are taking the number of transactions that contain both A and B and dividing it by the number of transactions that just contain A. The result gives us the conditional probability of B being bought when A is already in the cart. For example, if 60 out of 100 transactions that include bread also include butter, then confidence for bread to butter, bread and butter confidence is 60%. This tells us that when bread is purchased, butter is also purchased 60% of the time, which is a strong signal for product recommendation.

Confidence is particularly useful when deciding which item pairings are trustworthy enough to act on it in a recommendation system or promotional strategy. Another metric is lift. Lift helps us to understand how strong an association is between two items, not just whether they occurred together, but whether they occurred together more often than we would expect them by chance.

The formula is given as lift for A and A implies B is equal to confidence A implies B divided by support of B. Here what that means we take the probability that B is purchased when A is purchased that's confidence and divided by overall probability of B being purchased. If the result is greater than one it means that B is more likely to be bought when A is present than it is overall indicating a positive association. Generally, if lift is more than one, we say positive correlation A and B are bought together more often than by chance.

If it has a value of one, we say no correlation A and B occur independently. Less than one negative correlation buying A reduces the likelihood of B being bought, but practically lift will not be negative. Lift is a powerful metric for filtering out spurious associations.

While confidence may be high for certain rules, lift helps ensure that rule is truly meaningful not just a result of a popular item being purchased frequently. This makes lift essential for designing reliable and strategic recommendation rules. Let's now look at less common but insightful metric conviction.

While lift focusses on correlation, conviction helps us understand the strength of the implication. If A then B that's the implication. It's particularly useful where we care about how strongly A employs B rather than just their co-occurrence.

The formula is conviction of A employs B equal to $1 - \text{support of B}$ divided by $1 - \text{confidence of A employs B}$. If the confidence of a rule is high meaning B usually follows A, then the denominator becomes small pushing the conviction value higher. At the same time if B is rare overall the numerator becomes large. This combination signals a strong implication A almost always leads to B and B isn't commonly found without A. That's what conviction is.

Now that we have covered the key metrics let's walk through the overall process of how association rule mining works in market basket analysis. The first step is to extract frequent item sets. This means we can scan the data set to find combinations of item that occurred together frequently based on a minimum support threshold.

For instance if bread butter appears at in at least five percent of all transactions it qualifies as a frequent item set. Once we have these item sets, we move to generate rules. For each item set we create logical if then statements.

For example if you if we know that A, B and C is a frequent item set we can form rules like whenever people buy A and B, they also buy C which means when people are buying A and B, they're also buying C. Next, we evaluate rules using the metrics we discussed support confidence lift and optionally we use conviction as well. These help us assess how frequent how reliable and how meaningful each rule is. Finally we apply thresholds not all rules are useful so we filter them based on minimum confidence and lift values ensuring that only the strongest and most relevant rules are used in recommendation engines.

This is end-to-end process which enables us to convert raw transactional data into actionable business insights powering personalised recommendations targeted marketing and smarter product bundling. Let's now break down the a priori algorithm which is one of the most widely used methods for mining association rules in market basket analysis. The process starts with the first step find frequent items.

Here the algorithm scans the data set to identify individual items that meet the minimum support threshold. These are items that occur frequently enough to be of interest. We use the threshold so that we can filter out such products which are rarely bought.

In the next step we generate candidate item set this means we combine the frequent individual items into larger combinations like pairs triplets and so on forming candidate for potential rules. However not all combinations are useful so in the prune infrequent item set step the algorithm eliminates any item set that don't meet the minimum support threshold. This pruning is core feature of a priori it avoids unnecessary computations by eliminating low support combinations early.

Finally we extract strong rules from the remaining frequent item sets these rules are evaluated using metrics like confidence lift and sometimes conviction to determine their reliability and usefulness. A priori is simple yet powerful and its iterative structure allows us to efficiently mine useful insights from even larger transactional data set one level at a time. Let's look at how market basket analysis is applied in real world retail settings.

These use cases show the business impact of association rule mining beyond just theory. The first use case is in e-commerce you have likely seen the line customer who bought

this also bought while shopping on amazon or Flipkart. This is powered by basket analysis by identifying which products are commonly purchased together online retailers can cross sell items boosting the average order value and improving customer satisfaction.

Next in grocery store market basket analysis is used to optimise product placement for example if data reveals that chips and cells are frequently bought together placing them side by side in store increase the chances of both being purchased. A smart data-backed way to increase sales without additional promotions. The third use case is bundling retailers create combo offers of kits say shampoo conditioner hair serum because these items are often bought in sequence by bundling them store drive multiple items purchase and even influence brand loyalty by locking in preferences through smart packaging.

These are just a few ways that insights from the market basket analysis translate into revenue generating strategies proving its value across both physical and digital retail environments. Let's now look at visual example to understand how market basket analysis work using real transactions. This table shows five transactions, t1 to t5, and whether certain items were purchased in each which are bread, milk, butter, eggs and coffee. Our check mark means that item was included in that transaction. By scanning through the rows we can start identifying frequent patterns, for instance, bread appears four out of five transactions. That's an 80 percent support - bread and butter appears together in t1 t2 t3 and t5 so bread butter has a support of 80 percent.

Now let's talk confidence. Suppose we want to know the rule - if bread then butter. That's a rule. Out of the four transactions where bread was bought - all four also included butter, giving us 100 confidence for the rule, bread implies butter. You can also spot other interesting associations like milk and eggs in t4, coffee co-occurring with x in t3 and t4. This example shows how a binary transaction matrix can be transformed into meaningful association rules using metrics like support, confidence and lift and forms the basis for building intelligent recommendation systems and marketing strategies. Remember most of the implementation asks you to provide data in this particular format. You provide the transaction number and each transaction you provide, what items have been bought as a boolean values, true or false. As with any analytical techniques market basket analysis comes with both strength and limitations.

Let's take a closer look at both starting with strengths market basket analysis produces simple interpretable rules. Like if a customer buys bread, they are likely to buy butter. These rules are easy for business items. To be understood and teams to understand and act upon it doesn't require users profiles or personal data. Instead it works purely on transactional behaviour, making it privacy friendly and widely applicable it's extremely effective for cross-selling helping retailers surface product pairings that increase average basket size. Finally it is computationally efficient, especially with algorithms like apriori and FP growth. That's another one that prune the search space smartly. However there are some limitations. It's not personalised - the same recommendation applies to every user regardless of their past preferences or demographics. It can struggle with data sparsity in cases where there are too many unique items and not enough overlapping transactions to form strong rules. It misses context for example it does not account for the time of the day customer intent or external influence like promotions. Lastly it only looks at item presence not how much customer liked or disliked the product; it doesn't consider ratings or sentiment just binary inclusion in a transaction.

So while it is powerful for uncovering product relationship it's based used in combination with other methods like collaborative filtering or personalisation models to create richer, more adoptive recommendation system.

So that was your market basket analysis. Let's see how we implement this on the data. First, we are importing data set. I'll tell you what the data is all about. We are importing libraries that we need - import pandas as pd. Then from mlx10 a library. We are seeing frequent patterns import apriori and association rules. From mlx10 preprocessing we are importing transaction encoder, but without transaction encoder also we can work. Then we are reading the data pd.readcsv retail data csv and here is the snapshot of the data. We have the invoice number, the stock code and what's the description, what product you have bought, what's the quantity you have purchased invoice date unit price the customer id and the country. I understand over here we have the same invoice number. Over here it means we have stored the data in this particular fashion. I have these three items bought in a single transaction and we are storing it like this: invoice number is same, stock code changes, product description will change, different quantity and then unit price and all the details. So same invoice number will be repeating multiple times because you are buying multiple products in the same invoice.

Then we do some basic data cleaning, like if invoice number and description is missing - if any of the value is missing in either of these columns, delete the entire row. We are also removing cancel transaction - it starts with the letter c invoice number, if it starts with the letter c it means they are the cancel transaction, so we are removing that. And then we are removing negative or zero quantity. So we have done some basic data cleaning.

Then we create the basket format. As I mentioned to you earlier, we need the data in a format which says if an item has been purchased in the given transaction or not. So we are creating a basket here using pivot table. On the index side we are taking invoice number, description will go as a column. Values we are taking quantity and we are summing it up so what we will get is as a data frame where I will have invoice number as my index, I will have column as the product name, so product a product b product c. And under each invoice what's the quantity they have bought it will come over here.

But then a lot of products which will be there which is not bought in a particular transaction. So it will write NA over there. So instead of NA we are writing zero, and that's what we will get over here. Then as I mentioned earlier what we need is simply whether the product is part of the transaction or not. Nothing else. So we are converting quantities into one zero, whether it's present or not in the transaction. So the quantity is more than zero, we were writing as one; means that particular item has been purchased, else we are writing zero, means that item has not been purchased. So now this gives us the data that we are looking for.

The next we are applying apriori algorithm to get frequent item sets where we will provide that data so that we have created by mapping it to one zero. We provide minimum support over here because if I do not, then it will consider all the products even



if it has been bought only once in the entire lifetime of the store. We don't want too many rules; we want only some of the rules which are very important. So we give a threshold that if you have at least five percent of the transactions where the product has been sold then we'll consider that. And you can play with this number - if you increase it then probably, you'll have lesser number of products.

The next we are generating association rules. We are saying rules is equal to association rules. Frequent item set that we have created metric would be left minimum threshold would be one. Again you can play with the minimum threshold. And then we say rules we get the rules we are sorting the rules by the lift and here are first five rules that we have got. It says when people are buying you can read any of the rule like this when people are buying pack of six skull paper skull paper cups, they're also buying pack of skull paper plates. And for that rule it tells me what's the support, what's the confidence and what's the lift. So that's what we have got over here. And we have sorted it in descending order of lift. So most prominent rules they come on the top and then what you can do is you can pick up top five, top ten rules and you can implement in your business. So that's how you do association rule mining.